PROJECT KICK-OFF · APRIL 8, 2026

Conscia: Beyond Traditional PIM

A governed semantic foundation and real-time orchestration engine for complex, configurable commerce

POWERED BY: DX GRAPH + DX ENGINE

Conscia × Lovesac Joint Kick-off

Today's Agenda

01 **Problem Statement** — Recap of the challenges we're here to solve

02 **Meet the Teams** — Introductions and roles

03 **Conscia Platform Overview** — How DX Graph and DX Engine address Lovesac's needs

04 **Proposed Architecture** — Where Conscia sits within your existing stack

05 **Data Model Deep-Dive** — Collections, hierarchies, taxonomies and relationships

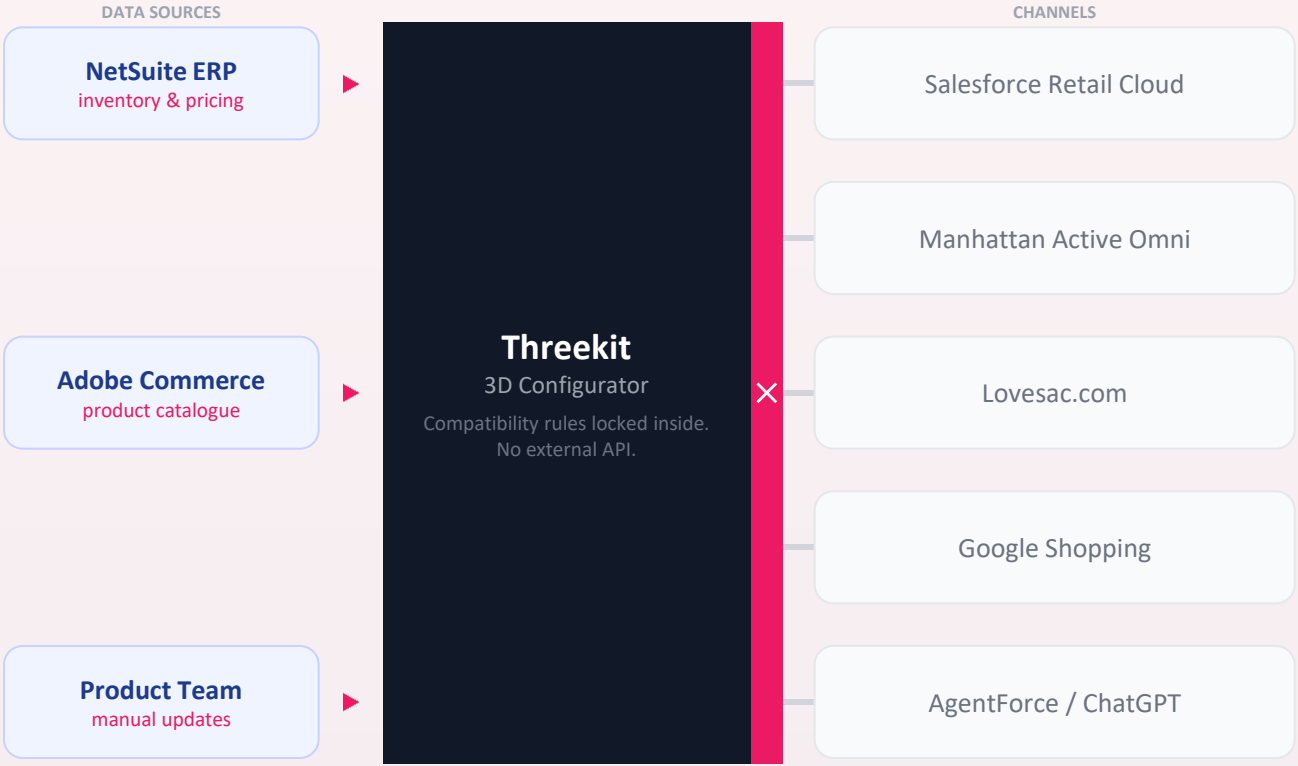
06 **Early Findings from Your Data** — Initial analysis from the pre-kickoff review

07 **Scope & Timeline** — Phase 1 scope and implementation plan

08 **Success Metrics & Ways of Working** — How we'll measure success and run the project

09 **Open Questions & Next Steps** — Items to resolve and agreed actions

Problem Statement — The Bottleneck



Problem: Compatibility intelligence is locked in one tool, and no downstream channel can query it directly.

(See Appendix for more detail)

Problem Statement — What Lovesac Can’t Do Today

01 You can’t **Ask “What goes with this Sactional cover?”**

Compatibility rules live only inside Threekit. No system can query them programmatically — so every channel has to hard-code logic or send the customer back to the web configurator.

02 You can’t **Serve product data to AgentForce or ChatGPT**

AI agents need a queryable API. Threekit has no MCP endpoint. Any agentic commerce experience requires custom middleware that duplicates and quickly drifts from the source of truth.

03 You can’t **Keep pricing and inventory in sync across channels**

NetSuite is the master for both pricing and inventory, but neither feeds the Threekit configurator in real time. Associates and customers frequently see conflicting data depending on which touchpoint they use.

04 You can’t **Launch a new sales channel without rebuilding logic**

Each new touchpoint (kiosk, app, Google Shopping) requires its own integration. There is no shared experience API — so every channel launch restarts the engineering effort from scratch.

05 You can’t **Understand how customers actually configure products**

Configuration session data is trapped inside Threekit. There is no way to extract patterns — which cover combinations are popular, where customers drop off, what drives conversion.

Meet the Teams

Conscia

M

Morgan · Project Manager
morgan@conscia.ai

S

Sana · Product Owner
sana@conscia.ai

L

Laurel · Sr. Solution Engineer
laurel@conscia.ai

D

David · Solution Architect
david@conscia.services

Lovesac

A

Amelia Jacobs · VP, Customer Experience & IT
ameliajacobs@lovesac.com

T

Tate Swanson · Sr. Domain Data Architect
tateswanson@lovesac.com

Conscia Platform Overview

01 · DX GRAPH

Semantic* Foundation

- Single SKU Collection unifying all brands, lines, and component roles
- Dynamic Relationships: COMPATIBLE_WITH, BUNDLES, CONTAINS, MAPS_TO
- Up to 3 parallel Taxonomy Collections via MAPS_TO
- Version-controlled, governed product data

One model. One source of truth.

02 · DX ENGINE

Orchestration

- Real-time orchestration of business rules across channels
- Channel-aware Experience Rules (e.g. Outdoor restrictions, Quickship eligibility)
- Deterministic logic + AI-assisted decisioning where it adds value
- Zero-code configuration through Conscia UI

Rules in one place — not duplicated per channel.

03 · EXPERIENCE APIs

Experience APIs

- One Experience API serving POS, web, mobile, agents, and feeds
- Sub-second responses at the edge
- Native MCP endpoint — agent-ready by design
- No frontend glue code; no per-channel reimplementation

New channels go live in days, not quarters.

*A semantic model stores facts *and* the meaningful connections between them: a Cover *belongs to* the 6 Series, *is compatible with* these Inserts, *is part of* these Bundles, *maps to* multiple channel taxonomies. Relationships are first-class, not query-time joins.

Proposed Architecture Fit

Two complementary platform capabilities that together make Conscia the governed product master for Lovesac.

DX GRAPH

Governed Product Data Layer

"What is true?"

- **Unified SKU Collection** — Sactional, Sac, Snugg, and Accessories — ~13,700 SKUs in one model
- **Cover ↔ Insert compatibility** encoded as a Dynamic Relationship (no manual tagging — covers all ~10,800 Cover SKUs automatically)
- **Up to 3 parallel Taxonomies** — internal ops, Lovesac.com, and marketplace feeds
- Version-controlled and audit-ready

DX ENGINE

Real-Time Orchestration Layer

"What should happen?"

- **Config Validation** — does this combination obey all the rules?
- **Wizard-Step Options** — what can the customer pick next, given what they've already chosen?
- **Config ID Creation** — durable identifier for a saved configuration
- **BOM Explosion** — turn a finished good into shippable components, shaped per channel (POS / OMS / warehouse pick list)
- One Experience API serving POS, web, agents, and feeds

End-to-End Architecture

SOURCE SYSTEMS

NetSuite
ERP
Financial master for pricing, cost & SKU lifecycle. Source of record for commercial attributes.

Adobe Commerce
ECOMMERCE
Web-facing product content, merchandising & storefront metadata.

Threekit
VISUAL CONFIGURATOR
Compatibility rules migrating to DX Graph.

INTEGRATION HUB

Celigo
INTEGRATION PLATFORM
Connects NetSuite, Adobe Commerce, Manhattan Active Omni & Conscia Layer via scheduled and event-driven flows.

CONSCIA PLATFORM

DX Graph
GOVERNED PRODUCT MASTER
Semantic data layer & system of record for configurable products. Holds unified SKU collection, compatibility relationships, taxonomy hierarchies & governed entities.

DX Engine
REAL-TIME ORCHESTRATION
Experience API: config validation, wizard steps, Config IDs, BOM explosion & channel feeds. MCP-exposable for agentic commerce.

DOWNSTREAM

Manhattan Active Omni (MAO)
OMS
Order master; receives exploded BOMs from DX Engine for fulfilment.

Lovesac.com
STOREFRONT
Adobe Commerce; consumes validated Config IDs via Experience API.

Salesforce Retail Cloud
POS
In-store config validation & Config ID lookup at the register.

CHANNELS & AGENTS

POS Terminals
IN-STORE

Lovesac.com
WEB

Mobile Apps

AgentForce
AI AGENT

Google Shopping
3RD-PARTY AGENTIC

ChatGPT / 3rd-Party AI
EMERGING AI

System names and integration paths from Lovesac Enterprise Applications Architecture, Target State, January 2026.

Collections, Hierarchies, Taxonomies

How DX Graph models Lovesac's configurable catalogue — Part 1: The Model

HIERARCHIES

Sactionals
› Covers
› 5 Series

SKU Collection · ~13,700 records

SKU ID	Line	Product	Color / Fabric
SAC-5S-COV	Cover	Seat	Charcoal Chenille
SAC-5S-INS	Insert	Seat	Standard Fill
SAC-6S-COV	Cover	Side	Oatmeal Linen
SAC-FG-3S	Bundle record	3-Seat Bundle	1:M Lookup → component Covers + Inserts in SKU Collection

One Collection · Four relationship types · Three parallel taxonomies

RELATIONSHIPS

- **COMPATIBLE_WITH** *Dynamic*
  Cover ↔ Insert (Product + Size)
- **BUNDLES** *1:M Lookup*
  Bundle record → component Covers + Inserts in SKU Collection
- **CONTAINS** *1:M*
  Composition relationships
- **MAPS_TO** *cross-collection*
  SKU → Taxonomy node

TAXONOMIES

- Internal Ops
- Lovesac.com
- Marketplace

via MAPS_TO

Collections

The foundation. Typed records with a defined schema. Lovesac's SKU Collection holds 28,089 records across all brands and lines.

Relationships

Connect records together. 1:1 or 1:M lookups for explicit links; Dynamic relationships use rule expressions — how Cover ↔ Insert compatibility is encoded.

Hierarchies

Special collections where records have a parent record, forming a tree. Paths recalculate automatically when records move.

Taxonomies

Sit on top of Collections to tag records dynamically by rule. Lovesac uses three in parallel — Internal Ops, Lovesac.com, and Marketplace — so the same SKU is classified differently per channel.

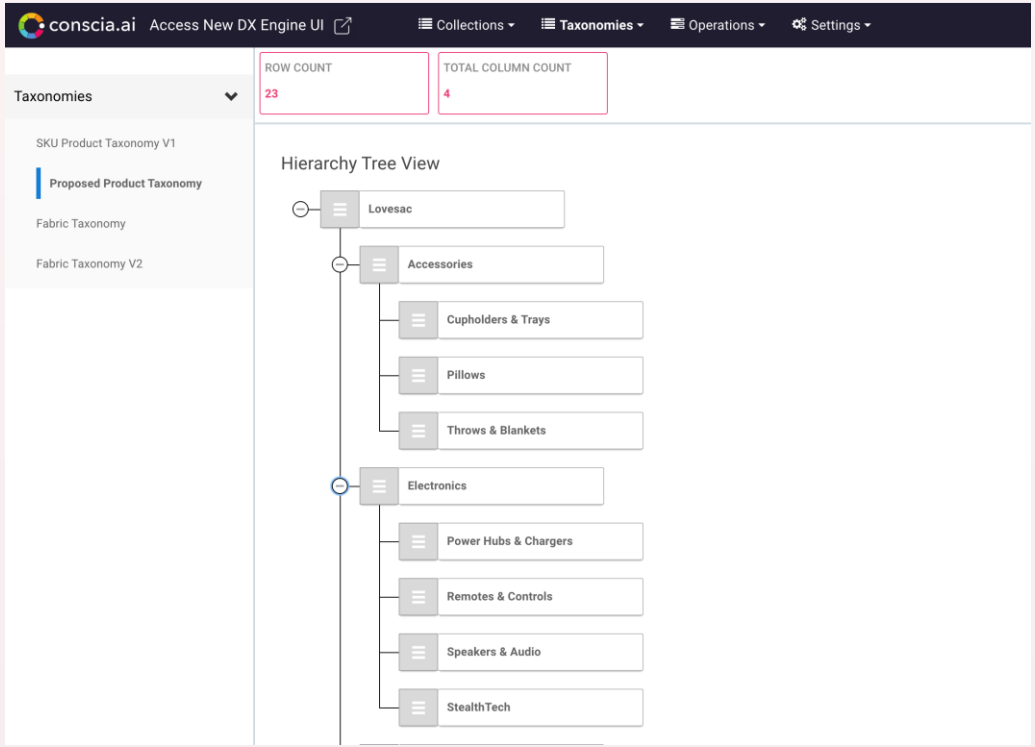
Collections, Hierarchies, Taxonomies

Demo of compatibility rules and data model in a sample hierarchy using Lovesac's data and Conscia's tooling — Part 2: Live in Conscia

GRAPH **DX Graph — Product Model**

DEMO

- *SKU Collection: ~13,700 records ingested from NetSuite*
- *Matrix items exploded into individual variant nodes*
- *Each node carries colour, cover, configuration attributes*
- *Relationships typed: COMPATIBLE_WITH, BUNDLES, CONTAINS*



Early Findings from Your Data

~13,700

Active SKUs analysed
across the full Lovesac catalogue

BREAKDOWN

~10,800 Sactional Covers
Covers · Inserts · other Sactional

~1,700 Sac & Snugg

~1,200 Accessories & spare parts

Across 3 brands: Sactional · Sac · Snugg

CATALOG STRUCTURE

Compatibility is encodable from two fields

Across all ~11,500 Cover SKUs, Cover ↔ Insert compatibility resolves from Product == Product AND Sactional_Size == Sactional_Size. A single Dynamic Relationship in DX Graph replaces tens of thousands of manual mappings — and new SKUs are covered automatically.

DATA QUALITY

Inconsistent attribute naming across product families

Color, fabric, and dimension fields use different naming conventions across Sactional, Sac, and Snugg. Recommend a normalisation pass during ingest to establish canonical attribute keys. Also observed: ~18 attribute fields per SKU; multiple legacy naming conventions; some product-line ambiguity (e.g. Squattoman: Sac or Ottoman?).

INTEGRATION READINESS

The big question is data access, not data shape

The product catalogue models cleanly. The open question is how data reaches Conscia from each source backend — API pull, file export via Celigo, or direct database. Confirming this for each of the 3 source systems is the first dependency on the timeline.

Project Scope — Phase 1

DX GRAPH
in scope

- DX Graph application provisioning and environment setup
- SKU Collection — all ~13,700 SKUs (Sactional, Sac, Snugg, Accessories) in one unified collection
- Configuration Collection for saved configs
- Up to 3 Taxonomy Collections
- Relationships: COMPATIBLE_WITH (Dynamic), BUNDLES (1:M), CONTAINS (1:M), MAPS_TO (1:M)
- Data ingest from up to 3 source backends via Jobs (via Celigo — recommended, see appendix)

✓ IN SCOPE

DX ENGINE POC
in scope

- Config Validation orchestration flow
- Visual Configurator wizard step options flow
- Config ID Creation flow (with API write-back to DX Graph)
- Config ID → BOM Explosion flow (POS / OMS / warehouse output shapes)
- Experience Rules for series, Outdoor restrictions, and channel-specific constraints

✓ IN SCOPE

OUT OF SCOPE — PHASE 1
not in Phase 1

- Lovesac.com frontend integration (handled by Lovesac team)
- Real-time inventory availability (depends on Q4 — Quickship resolution)
- Production deployment to AgentForce / ChatGPT / Google Shopping (POC only)
- Pricing engine integration (NetSuite continues as pricing master)
- Manhattan Active Omni / OMS integration (downstream of BOM explosion)
- PLM integration (TBD — depends on Q2)

✕ OUT OF SCOPE

Implementation Timeline

PHASE 1
Weeks 1–3
TBD

Foundation & Data

- Provision DX Graph environment
- Model SKU Collection (single collection, all ~13,700 SKUs)
- Model Configuration Collection
- Build Cover↔Insert COMPATIBLE_WITH Dynamic Relationship

PHASE 2
Weeks 4–6
TBD

Taxonomy & Ingest

- Model up to 3 Taxonomy Collections
- Map all SKUs via MAPS_TO
- Configure ingest Jobs from 3 source backends (via Celigo where applicable)
- Validate data completeness and AI-derived compatibility rules with Lovesac team

PHASE 3
Weeks 7–10
TBD

DX Engine POC

- Build Config Validation orchestration flow
- Build Wizard Step Options flow
- Build Config ID Creation flow (with DX Graph API write-back)
- Build BOM Explosion flow (POS / OMS / warehouse output shapes)

PHASE 4
Weeks 11–12
TBD

Testing & Handoff

- End-to-end integration testing across all flows
- Channel QA (POS via Salesforce Retail Cloud, Lovesac.com sandbox)
- Documentation and runbooks
- Knowledge transfer to Lovesac team

⚠ Phase 1 dependency
Source backend confirmation (Q1)

⚠ Phase 2 dependency
Hierarchy decision (Q3) + adjacency rules (Q5)

⚠ Phase 3 dependency
Quickship resolution (Q4) + PLM alignment (Q2)

✓ Phase 4 — clear to proceed
No blocking dependencies
Contingent on Phases 1–3 completing on schedule

Time estimates are indicative and will be confirmed once open questions are resolved per the dependency flags above.

Success Metrics & KPIs

Phase 1 Engagement Goals

- All ~13,700 SKUs ingested into a single DX Graph SKU Collection
- COMPATIBLE_WITH Dynamic Relationship live across all ~10,800 Cover SKUs
- Up to 3 Taxonomy Collections modelled and mapped via MAPS_TO
- All four DX Engine flows (Config Validation, Wizard Step Options, Config ID Creation, BOM Explosion) running end-to-end against Lovesac data

Business Outcomes

- Single source of truth for Sactional compatibility
- POS associates can validate any configuration in real time
- New channels (kiosk, agent, feed) reuse the same Experience API instead of rebuilding logic
- Foundation in place for AgentForce / ChatGPT / Google Shopping integration

How We'll Measure

- API response time: < 200ms p95 for configurator queries
- Zero data sync failures over a 30-day observation window
- All four DX Engine flows pass end-to-end UAT against Lovesac test cases
- Lovesac sign-off at each phase gate (4 gates over 12 weeks)

Latency and volume targets to be confirmed with Lovesac at first technical review.

Ways of Working

TEAM STRUCTURE

Role	Conscia	Lovesac
Executive Sponsor	Sana	Amelia Jacobs
Project Lead	Morgan	*TBC at kickoff*
Solution Architect	David	Tate Swanson
Solution Engineer	Laurel	*Data owner — TBC*
Platform Engineer	Harshil	*Tech lead — TBC*
QA / Testing	Laurel & Harshil	*TBC*

Weekly **Project Standup**
30 min · Both teams · Status, blockers, decisions

Bi-weekly **Steering Check-in**
60 min · Leads only · Scope, risks, milestones

As-needed **Technical Review**
Data model, API design, integration reviews

End of Phase **Phase Gate Review**
Demo, sign-off, and next phase kick-off

Ongoing **Slack / async comms**
Shared channel for day-to-day coordination

Open Questions

We've shared a full questions document with your team ahead of today (32 questions across 5 areas). This slide is the shape of that conversation — we'll go deeper on the items most likely to shape Phase 1.

12 Qs DATA

- *Where did each product file come from, and which is authoritative?*
- *Will product data flow via Celigo, or directly from source?*

6 Qs INTEGRATION

- *PLM (TBD) vs DX Graph — who is the product master?*
- *How does a validated config reach Manhattan Active Omni today?*

9 Qs CONFIG & RULES

- *Quickship — product type or live inventory?*
- *Adjacency rules — please validate our proposed table.*

3 Qs USER STORIES

- *What does the in-store POS configuration workflow look like today?*
- *Are there planned changes to the lovesac.com configurator UX?*

2 Qs PROJECT & GOVERNANCE

- *Who is the named Lovesac PM?*
- *What are the POS volume and latency expectations?*

Full document: **Questions_for_Lovesac_EXT_Conscia.docx** — sent April 2

Next Steps

Actions coming out of today's session — owners and target dates for both teams.

April 9	Both	Distribute kickoff notes, decisions log, and open-question owners
April 11	Lovesac	Confirm the 3 source backends — name, access method (API / file via Celigo / direct), refresh cadence
April 11	Lovesac	Confirm which of the 3 hierarchy proposals from the March 12 review will be adopted
April 14	Lovesac	Validate the proposed component adjacency table (resolves Q5)
April 9	Lovesac	Name the Lovesac Project Manager and Data Owner
April 11	Conscia	Provision DX Graph sandbox environment and share credentials
April 9	Conscia	Share the AI-derived compatibility rules document for product-team review
April 15	Both	First weekly project standup — review readiness, lock Phase 1 sprint plan

APPENDIX

APPENDIX

Reference material — Threekit & Conscia comparison and phased role transition.

Threekit & Conscia: Different Jobs, Different Tools

Dimension	Threekit	Conscia
Primary job	3D product visualisation & configuration	API-layer orchestration across all systems
Data stored	Visual assets, component rules, constraints	Product graph, business logic, channel rules
Who queries it	Lovesac.com configurator only	Any channel — web, app, kiosk, AI agent
How data flows	Push: Lovesac team uploads updates manually	Pull: real-time API calls, live sync
AI / agent access	Not designed for programmatic access	Native MCP endpoint; agent-ready by design
Where it lives	SaaS, Threekit-hosted	Orchestration layer above your existing stack
Phase 1 role	Continues as the rendering engine	Reads from Threekit, exposes data everywhere

Celigo and Conscia: Complementary Layers

Different categories of tool solving different problems in the Lovesac stack

Concern	Celigo (iPaaS)	Conscia (DX Graph + DX Engine)
Category	Integration Platform as a Service	Governed semantic layer + experience orchestration
Core job	Move records between SaaS apps with mapping and error handling	Hold the canonical product model and serve real-time channel APIs
Data pattern	Batch and event-driven flows, source-to-destination	Unified graph with relationships (COMPATIBLE_WITH, BUNDLES, CONTAINS)
Latency	Throughput-optimised: scheduled or webhook	Sub-second synchronous, on the critical path of POS and web
Rules	Flow-level branching for integration builders	Business-user Experience Rules: priority, A/B, segmentation
Frontend	None — frontends call multiple flows or build their own BFF	Single Experience API call returns unified, channel-shaped JSON
Agentic / MCP	MCP server exposes integration flows	MCP server exposes governed semantic capabilities
Where it shines	NetSuite ↔ Adobe Commerce ↔ Manhattan Active Omni ↔ Salesforce plumbing	Config validation, BOM explosion, wizard steps, agentic channels

The Sactional SKU-explosion problem is a model and orchestration problem, not a transport problem.

Celigo moves data between systems. Conscia gives that data meaning and serves it in real time.

How Conscia Integrates with Celigo

Three patterns, one end-state architecture

A. INGEST

RECOMMENDED

Celigo feeds Conscia

- Celigo's existing NetSuite item/matrix/kit exports also write to Conscia ingest endpoint
- Reuses existing saved searches, custom fields, and field mappings
- JSON webhook or SFTP — nightly delta + on-demand full refresh

Why: don't rebuild work Celigo is already doing well

B. READ PATH

POS calls Conscia direct, Lovesac.com calls Experience API & Agent shopping calls via MCP

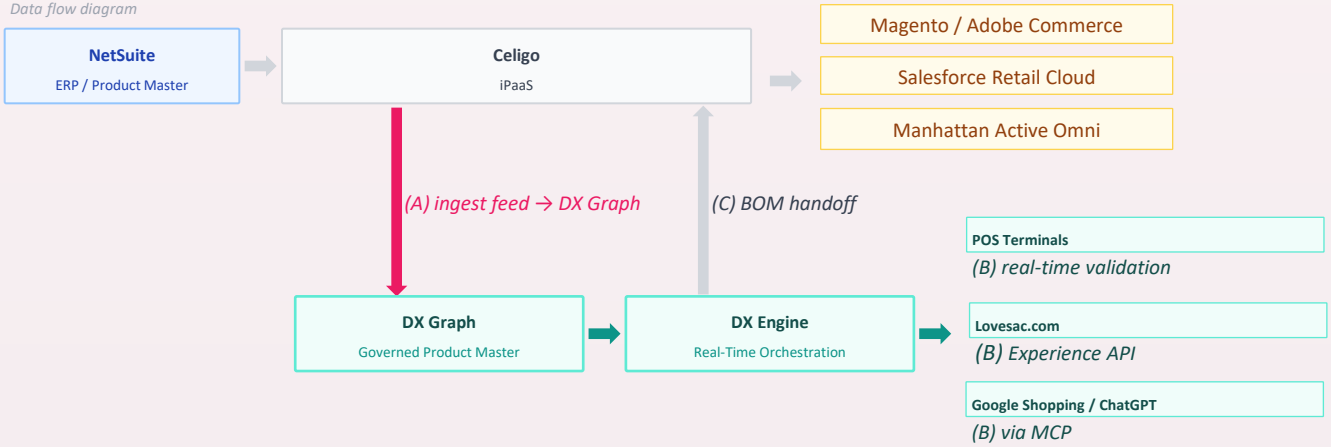
- Salesforce Retail Cloud calls DX Engine Experience API directly for real-time config validation
- Celigo is not on this path — wrong latency profile for synchronous checkout

Phase 1 POC scope

C. BOM HANDOFF

Conscia hands off via Celigo

- DX Engine produces exploded BOM from a Config ID
- Hands off to existing Celigo flow talking to Manhattan Active Omni

Conscia = config brain; Celigo = logistics plumbing

Data flow diagram